

Análisis Avanzado de Datos

Sesión 7 y 8: Datos No Estructurados y Modelos de Texto

Nicolás López

Primer semestre de 2026

- 1 Puente: de variables latentes a texto
- 2 Introducción: datos no estructurados
- 3 Análisis textual: Entender
- 4 Análisis textual: Estandarizar
- 5 Análisis textual: Describir
- 6 Análisis textual: Modelar con LDA
- 7 LDA Supervisado (sLDA)
- 8 Más allá de BOW: modelos de lenguaje
- 9 Entrenamiento con gradiente descendente

10 BERTopic: modelado de temas con embeddings

11 Comparaciones y cierre

12 Síntesis y cierre

13 Referencias

Puente: de variables latentes a texto

Lo que hemos construido (Sesiones 1–6)

Las primeras seis sesiones cubrieron datos **estructurados**: matrices $X \in \mathbb{R}^{n \times p}$ donde cada fila es un individuo y cada columna una variable numérica o categórica.

La Sesión 6 introdujo las **variables latentes**:

- **EM/Mixtura**: la membresía $\Delta_i \in \{0, 1\}$ de cada observación a un grupo no observable.
- **PCA**: scores latentes continuos en $k \ll p$ dimensiones.
- **ACM**: scores latentes para variables categóricas.

El salto de hoy

El texto no es una matriz X : es una secuencia de palabras. Para modelarlo necesitamos extender la idea de variable latente al caso de datos **no estructurados**.

LDA como EM aplicado a texto

La conexión entre LDA y lo visto en la Sesión 6 es directa:

Sesión 6 – Mixtura gaussiana	Sesión 7 – LDA
Observación y_i	Palabra w_{dn}
Componente k	Tema k
Membresía $\Delta_i \in \{0, 1\}$	Asignación $z_{dn} \in \{1, \dots, K\}$
Peso π_k	Proporción de temas θ_d
Media μ_k	Distribución sobre palabras β_k
Responsabilidad $\hat{\gamma}_i$	Prob. posterior de tema

Ambos modelos usan EM: el Paso E estima la probabilidad de membresía (al grupo / al tema), y el Paso M actualiza los parámetros de las distribuciones.

Puente: de
variables
latentes a
texto

**Introduccion:
datos no es-
tructurados**

Analisis
textual:
Entender

Analisis
textual:
Estandarizar

Analisis
textual:
Describir

Analisis
textual:
Modelar con
LDA

LDA
Supervisado
(sLDA)

Mas alla de
BOW:
modelos de
lenguaje

Entrenamiento
con
gradiente
descendente

Introduccion: datos no estructurados

Datos estructurados vs. no estructurados

Puente: de
variables
latentes a
texto

Introducción:
datos no es-
tructurados

Análisis
textual:
Entender

Análisis
textual:
Estandarizar

Análisis
textual:
Describir

Análisis
textual:
Modelar con
LDA

LDA
Supervisado
(sLDA)

Más allá de
BOW:
modelos de
lenguaje

Entrenamiento
con
gradiente
descendente

Información estructurada

UE	V_1	...	V_p		
1	UE	V_1	...	V_p	
2	1	UE	V_1	...	V_p
3	2	1			
...	3	2			
n	...	3			
	n	...			
		n			



Visualización



Cuantificación

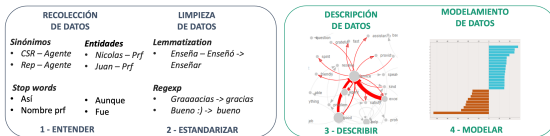
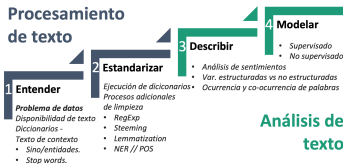


Modelamiento

Información no estructurada (textual)



En este caso, quisieramos organizar esta informacion en una biblioteca (no supervisado), o usarla para recomendar nuevas lecturas (supervisado).



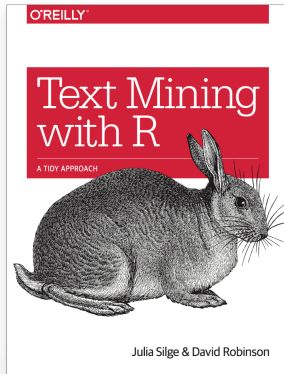
Welcome to Text Mining with R

This is the [website](#) for *Text Mining with R*. Visit the [GitHub repository](#) for this site, find the book at O'Reilly, or buy it on Amazon.

This work by Julia Silge and David Robinson is licensed under a Creative Commons Attribution-NonCommercial-ShareAlike 3.0 United States License.



[Preface »](#)



Disponible en: <https://www.tidytextmining.com/>

Puente: de
variables
latentes a
texto

Introduccion:
datos no es-
tructurados

Análisis
textual:
Entender

Análisis
textual:
Estandarizar

Análisis
textual:
Describir

Análisis
textual:
Modelar con
LDA

LDA
Supervisado
(sLDA)

Más allá de
BOW:
modelos de
lenguaje

Entrenamiento
con
gradiente
descendente

Analisis textual: Entender

Preguntas clave antes de modelar

- ¿De donde se originan los datos textuales?
- ¿Cuales son los criterios de inclusion y exclusion?
- ¿Representan una muestra de una coleccion o es una poblacion de textos?
 - ¿Es representativa?
 - Quiza responden unicamente clientes insatisfechos.
 - El texto proviene de una muestra por conveniencia.
- ¿Que informacion estructurada existe que pueda explicar el comportamiento visto marginalmente del texto?

Puente: de
variables
latentes a
texto

Introduccion:
datos no es-
tructurados

Análisis
textual:
Entender

Análisis
textual:
Estandarizar

Análisis
textual:
Describir

Análisis
textual:
Modelar con
LDA

LDA
Supervisado
(sLDA)

Mas alla de
BOW:
modelos de
lenguaje

Entrenamiento
con
gradiente
descendente

Contexto y diccionarios

El entendimiento permite acotar el problema e iniciar una depuración específica del contexto:

Sinónimos: - Mesero / Garzón / Waiter / Mozo. - Reclamo / Queja / PQR / Ticket.

Entidades: - Restaurante La Pinta. - Mercado libre.

Stopwords: - “restaurante” (si el objetivo no es identificar locales). - “reclamo” (si es la única razón de un corpus de quejas).

Se recolectan expresiones y palabras **en contexto** determinantes en nuestro conjunto de datos.

Tipos de diccionarios

(1) Sinónimos: conocimiento de sinónimos para una palabra relevante en el contexto de interés. Puede requerir consulta con la persona experta del dominio.

- Mesero / Garzón / Waiter / Mozo.

(2) Entidades (NER): para reseñas de restaurantes, cada nombre de local puede cruzarse con un catálogo estructurado. Este match permite identificar establecimientos.

(3) Stopwords: en ese mismo caso, los nombres de los restaurantes probablemente sean una stopword a retirar. También existen listas genéricas pre-establecidas (usar con cuidado: pueden eliminar palabras relevantes para el contexto específico).

Puente: de
variables
latentes a
texto

Introducción:
datos no es-
tructurados

Análisis
textual:
Entender

**Análisis
textual:
Estandarizar**

Análisis
textual:
Describir

Análisis
textual:
Modelar con
LDA

LDA
Supervisado
(sLDA)

Más allá de
BOW:
modelos de
lenguaje

Entrenamiento
con
gradiente
descendente

Análisis textual: Estandarizar

Contamos con resenas de restaurantes con ortografia y vocabulario variados.
`unnest_tokens` convierte a formato tidy: **una fila por token**.

```
library(tidytext); library(tidyverse)
data0 <- c(
  "El Mesero de La Pinta fue excelente!!!",
  "MESERO pesimo, nunca trajo la cta. No vuelvo",
  "El waiter tardo 40 min pero la comida 10/10",
  "el garzon del local de chapinero es mala onda")

datal <- tibble(resena = data0) %>%
  mutate(ID = row_number()) %>%
  unnest_tokens(word, resena)
head(datal, 8)
```

```
## # A tibble: 8 x 2
##       ID word
##   <int> <chr>
## 1     1  el
## 2     1 mesero
## 3     1  de
## 4     1  la
## 5     1 pinta
## 6     1  fue
## 7     1 excelente
## 8     2 mesero
```

`unnest_tokens` reduce a minusculas y retira puntuacion.

Aplicacion del diccionario de sinonimos

Puente: de
variables
atentes a
texto

Introduccion:
datos no es-
tructurados

Análisis
textual:
Entender

Análisis
textual:
Estandarizar

Análisis
textual:
Describir

Análisis
textual:
Modelar con
LDA

LDA
Supervisado
(sLDA)

Mas alla de
BOW:
modelos de
lenguaje

Entrenamiento
con
gradiente
descendente

A traves de un `left_join` se aplica la estandarizacion:

```
dict_syn <- tibble(  
  word      = c("waiter", "garzon", "mozo"),  
  Palabra1 = c("mesero", "mesero", "mesero"))  
data1 %>% left_join(dict_syn, by = "word") %>% head(9)
```

```
## # A tibble: 9 x 3  
##       ID word      Palabra1  
##   <int> <chr>    <chr>  
## 1     1 el      <NA>  
## 2     1 mesero  <NA>  
## 3     1 de      <NA>  
## 4     1 la      <NA>  
## 5     1 pinta   <NA>  
## 6     1 fue      <NA>  
## 7     1 excelente <NA>  
## 8     2 mesero  <NA>  
## 9     2 pesimo   <NA>
```

Las palabras sin match conservan NA en Palabra1. La logica es: si hay match, usar la forma canonica; si no, conservar la palabra original.

Reconstrucción del texto estandarizado

Puente: de
variables
atentes a
texto

Introducción:
datos no es-
tructurados

Análisis
textual:
Entender

Análisis
textual:
Estandarizar

Análisis
textual:
Describir

Análisis
textual:
Modelar con
LDA

LDA
Supervisado
(sLDA)

Más allá de
BOW:
modelos de
lenguaje

Entrenamiento
con
gradiente
descendente

```
data12 <- data1 %>%  
  left_join(dict_syn, by = "word") %>%  
  mutate(word0 = ifelse(is.na(Palabra1), word, Palabra1)) %>%  
  group_by(ID) %>%  
  summarize(resena0 = paste(word0, collapse = " "))  
data12
```

```
## # A tibble: 4 x 2  
##       ID resena0  
##   <int> <chr>  
## 1     1 1 el mesero de la pinta fue excelente  
## 2     2 2 mesero pesimo nunca trajo la cta no vuelvo  
## 3     3 3 el mesero tardo 40 min pero la comida 10 10  
## 4     4 4 el mesero del local de chapinero es mala onda
```

Expresiones regulares como alternativa

El procedimiento funciona, pero las variantes tipográficas (p.ej. “garzón”, “garzones”, “Garzon”) dificultan el proceso. Las **expresiones regulares** son más flexibles:

```
data1 <- sub(
  "[Gg]ar[zs] [oó]n(es)? [Ww]aiter | [Mm]ozo",
  "mesero",
  data0)
tibble(resena0 = data1)
```

```
## # A tibble: 4 x 1
##   resena0
##   <chr>
## 1 El Mesero de La Pinta fue excelente!!!
## 2 MESERO pesimo, nunca trajo la cta. No vuelvo
## 3 El mesero tardó 40 min pero la comida 10/10
## 4 el mesero del local de chapinero es mala onda
```

Las regex son especialmente útiles cuando el mismo concepto tiene muchas variantes morfológicas o de codificación.

Pasos adicionales de estandarizacion

Puente: de
variables
latentes a
texto

Introduccion:
datos no es-
tructurados

Análisis
textual:
Entender

Análisis
textual:
Estandarizar

Análisis
textual:
Describir

Análisis
textual:
Modelar con
LDA

LDA
Supervisado
(sLDA)

Más allá de
BOW:
modelos de
lenguaje

Entrenamiento
con
gradiente
descendente

- La estandarizacion a traves de diccionarios continua para entidades y stopwords.
- Las entidades pueden encontrarse tambien por modelos pre-entrenados (*transfer learning* con datos etiquetados).
- Se continua con **lematizacion y/o stemming**: “correr”, “corriendo”, “corrio” → “corr”. Puede basarse en diccionarios o modelos de lenguaje.
- Algunos procesos se benefician de **POS tagging** (asignacion de palabras a su categoria gramatical: sustantivo, verbo, adjetivo. . .) para depuracion informativa del texto.

¿Tiene sentido limpiar cuando usamos embeddings?

La respuesta depende del modelo:

Modelo	Limpieza necesaria	Razon
BOW / TF-IDF / LDA	Exhaustiva	Cada forma es una columna distinta en la DTM
Word2Vec clásico	Basica (minúsculas, ruido)	Variantes morfológicas caen cerca en el espacio vectorial
Transformers (BERT)	Casi ninguna	El tokenizador de subpalabras maneja morfología, mayúsculas y signos

Consecuencia práctica

Con BERT, “corriendo”, “corrio” y “correr” son subpalabras relacionadas que el modelo entiende sin lematización manual. Las *stopwords* incluso aportan contexto sintáctico útil para la atención.

Puente: de
variables
latentes a
texto

Introduccion:
datos no es-
tructurados

Analisis
textual:
Entender

Analisis
textual:
Estandarizar

**Analisis
textual:
Describir**

Analisis
textual:
Modelar con
LDA

LDA
Supervisado
(sLDA)

Mas alla de
BOW:
modelos de
lenguaje

Entrenamiento
con
gradiente
descendente

Analisis textual: Describir

Datos de ejemplo: Sherlock Holmes

Descargamos *The Adventures of Sherlock Holmes* (A. C. Doyle, 1892) desde el Proyecto Gutenberg. Colección de 12 relatos de misterio: vocabulario rico y temática reconocible.

```
library(readr)
url <- 'https://www.gutenberg.org/cache/epub/1661/pg1661.txt'
sherlock_text <- read_lines(url)

library(dplyr)
sherlock_raw <- tibble(gutenberg_id = 1661, text = sherlock_text)
dim(sherlock_raw)
```

```
## [1] 12310      2
head(sherlock_raw, 4)
```

```
## # A tibble: 4 x 2
##   gutenberg_id text
##   <dbl> <chr>
## 1      1661 "The Project Gutenberg eBook of The Adventures of Sherlock Holme-
## 2      1661 "
## 3      1661 "This eBook is for the use of anyone anywhere in the United Stat-
## 4      1661 "most other parts of the world at no cost and with almost no res-
```

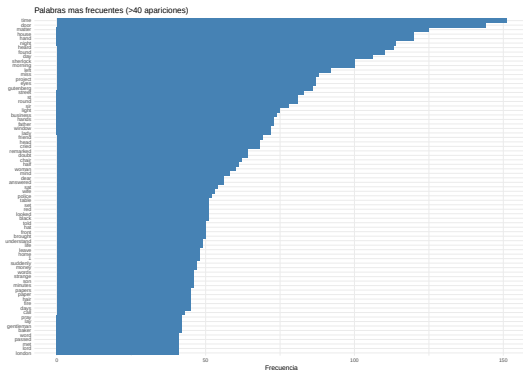

Tokenización y limpieza

```
custom_stop <- tibble(  
  word = c("holmes", "watson", "said", "mr", "upon"))  
  
tidy_sherlock <- sherlock_raw %>%  
  filter(text != "") %>%  
  filter(row_number() >= 10) %>%  
  mutate(n_line = row_number()) %>%  
  unnest_tokens(word, text) %>%  
  anti_join(stop_words, by = "word") %>%  
  anti_join(custom_stop, by = "word")
```

`anti_join(stop_words)` elimina palabras funcionales. Se agregan como stopwords “holmes” y “watson” porque aparecen en casi todos los párrafos y no aportan información diferenciadora entre pasajes.

Palabras mas frecuentes (unigramas)

```
tidy_sherlock %>%  
  count(word, sort = TRUE) %>%  
  filter(n > 40) %>%  
  mutate(word = reorder(word, n)) %>%  
  ggplot(aes(n, word)) +  
  geom_col(fill = "steelblue") +  
  theme_minimal(base_size = 9) +  
  labs(y = NULL, x = "Frecuencia",  
       title = "Palabras mas frecuentes (>40 apariciones)")
```



Nube de palabras

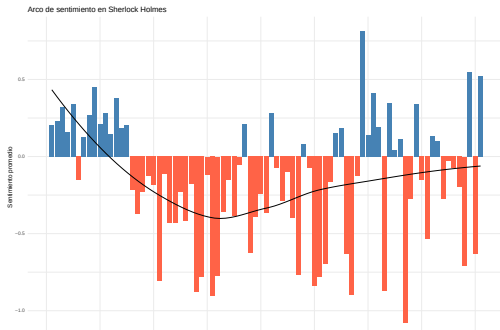
```
library(wordcloud)
tidy_sherlock %>%
  count(word) %>%
  with(wordcloud(word, n, max.words = 80,
                 colors = brewer.pal(8, "Dark2"),
                 scale = c(3.5, 0.4)))
```



Analisis de sentimientos: arco narrativo

Sentimiento promedio por seccion del libro (AFINN):

```
library(textdata)
afinn_dict <- get_sentiments("afinn")
sherlock_sent <- tidy_sherlock %>%
  left_join(afinn_dict, by = "word") %>%
  mutate(seccion = 1 + round(n_line / 120)) %>%
  group_by(seccion) %>%
  summarize(sent_prom = mean(value, na.rm = TRUE))
ggplot(sherlock_sent, aes(x = seccion, y = sent_prom)) +
  geom_col(aes(fill = sent_prom > 0), show.legend = FALSE) +
  scale_fill_manual(values = c("tomato", "steelblue")) +
  geom_smooth(se = FALSE, color = "black", linewidth = 0.6) +
  theme_minimal(base_size = 9) +
  labs(y = "Sentimiento promedio", x = "Seccion del libro",
       title = "Arco de sentimiento en Sherlock Holmes")
```



Palabras positivas v negativas mas frecuentes

```
tidy_sherlock %>%  
  left_join(afinn_dict, by = "word") %>%  
  mutate(signo = case_when(  
    value > 0 ~ "Positivo",  
    value < 0 ~ "Negativo",  
    TRUE ~ "Neutro")) %>%  
  filter(signo %in% c("Positivo", "Negativo")) %>%  
  count(signo, word, sort = TRUE) %>%  
  group_by(signo) %>%  
  slice_max(n, n = 6) %>%  
  arrange(signo, desc(n))
```

```
## # A tibble: 12 x 3  
## # Groups:   signo [2]  
##   signo      word      n  
##   <chr>    <chr>  <int>  
## 1 Negativo miss      88  
## 2 Negativo cried     68  
## 3 Negativo doubt     64  
## 4 Negativo leave     48  
## 5 Negativo strange    46  
## 6 Negativo fire       45  
## 7 Positivo matter    125  
## 8 Positivo dear       56  
## 9 Positivo pray       42  
## 10 Positivo save      37  
## 11 Positivo strong     35  
## 12 Positivo hope      32
```

Observa la ambigüedad en contexto: “strange”, “dark”, “criminal” son negativos en el diccionario pero pueden ser puramente descriptivos en un relato de misterio.

Bigramas: texto en contexto

Puente: de
variables
latentes a
texto

Introducción:
datos no es-
tructurados

Análisis
textual:
Entender

Análisis
textual:
Estandarizar

Análisis
textual:
Describir

Análisis
textual:
Modelar con
LDA

LDA
Supervisado
(sLDA)

Más allá de
BOW:
modelos de
lenguaje

Entrenamiento
con
gradiente
descendente

```
sherlock_big <- tidy_sherlock %>%  
  group_by(n_line) %>%  
  summarize(frase_limpia = paste(word, collapse = " ")) %>%  
  unnest_tokens(bigrama, frase_limpia,  
    token = "ngrams", n = 2) %>%  
  filter(!is.na(bigrama))  
head(sherlock_big, 6)
```

```
## # A tibble: 6 x 2  
##   n_line bigrama  
##   <int> <chr>  
## 1      1 title adventures  
## 2      1 adventures sherlock  
## 3      2 author arthur  
## 4      2 arthur conan  
## 5      2 conan doyle  
## 6      4 release date
```

Bigramas mas frecuentes

```
sherlock_topbi <- sherlock_big %>% count(bigrama, sort = TRUE)  
head(sherlock_topbi, 12)
```

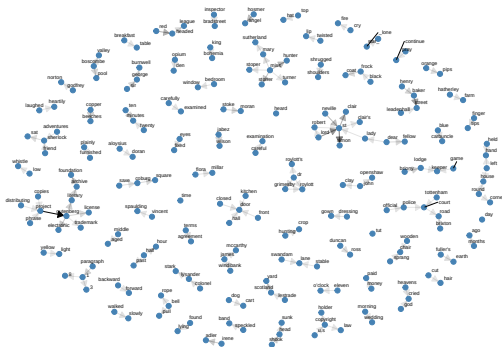
```
## # A tibble: 12 x 2  
##   bigrama          n  
##   <chr>          <int>  
## 1 project gutenber 71  
## 2 st simon        36  
## 3 lord st         28  
## 4 baker street    25  
## 5 red headed      20  
## 6 st clair        20  
## 7 miss hunter     18  
## 8 hosmer angel    16  
## 9 neville st     16  
## 10 miss stoner    14  
## 11 archive foundation 13  
## 12 gutenber electronic 13
```

Los bigramas revelan la estructura narrativa: personajes, lugares y acciones recurrentes del universo del detective.

Bigramas como grafo

```

library(igraph); library(ggraph)
bi_sep <- sherlock_big %>% separate(bigrama, c("word1", "word2"), sep = " ")
bi_graph <- bi_sep %>%
  count(word1, word2, sort = TRUE) %>%
  filter(n >= 4) %>% graph_from_data_frame()
a <- grid::arrow(type = "closed", length = unit(.12, "inches"))
ggraph(bi_graph, layout = "fr") +
  geom_edge_link(aes(edge_alpha = n), show.legend = FALSE,
    arrow = a, end_cap = circle(.06, "inches")) +
  geom_node_point(color = "steelblue", size = 3) +
  geom_node_text(aes(label = name), vjust = 1, hjust = 1,
    size = 2.5, repel = TRUE) + theme_void()
  
```



Metricas del grafo de bigramas

Puente: de
variables
latentes a
texto

Introduccion:
datos no es-
tructurados

Analisis
textual:
Entender

Analisis
textual:
Estandarizar

Analisis
textual:
Describir

Analisis
textual:
Modelar con
LDA

LDA
Supervisado
(sLDA)

Mas alla de
BOW:
modelos de
lenguaje

Entrenamiento
con
gradiente
descendente

El grafo de bigramas permite calcular metricas de red:

- **Grado de entrada/salida:** cuantas veces una palabra sigue o precede a otra.
- **Centralidad:** palabras que conectan multiples cadenas semanticas (“street”, “night”, “face”).
- **Clustering:** grupos de palabras que co-ocurren frecuentemente (subgrafos tematicos).
- **Distancia:** cuantos pasos separan dos palabras poco relacionadas.

Esto es la version textual de los kernels de grafo (Sesion 4): la estructura del texto como red.

Puente: de
variables
latentes a
texto

Introducción:
datos no es-
tructurados

Análisis
textual:
Entender

Análisis
textual:
Estandarizar

Análisis
textual:
Describir

**Análisis
textual:
Modelar con
LDA**

LDA
Supervisado
(sLDA)

Más allá de
BOW:
modelos de
lenguaje

Entrenamiento
con
gradiente
descendente

Análisis textual: Modelar con LDA

LDA: la idea central

El **Análisis de Dirichlet Latente (LDA)** asume:

- Los documentos están compuestos por múltiples **temas**: algunos temas se presentan más frecuentemente que otros para un documento dado.
- Los temas son colecciones de **palabras** asociadas: algunas palabras son más prevalentes que otras para cada tema dado.

LDA: la idea central

LDA y variables latentes (Sesión 6)

Los **temas** son variables latentes: no los observamos directamente, pero determinan las palabras que aparecen en cada documento. Exactamente la misma lógica que la membresía Δ_i en la mezcla gaussiana.

El proceso generativo: vision intuitiva

Puente: de
variables
latentes a
texto

Introduccion:
datos no es-
tructurados

Análisis
textual:
Entender

Análisis
textual:
Estandarizar

Análisis
textual:
Describir

Análisis
textual:
Modelar con
LDA

LDA
Supervisado
(sLDA)

Mas alla de
BOW:
modelos de
lenguaje

Entrenamiento
con
gradiente
descendente

Imagine que un autor escribe un documento siguiendo estos pasos:

Paso 1: el autor decide con que proporcion va a tratar cada tema. “Este articulo sera 60% sobre deportes, 30% sobre politica, 10% sobre economia.”
Esto es el vector de proporciones θ_d .

Paso 2: para escribir cada palabra, el autor: (a) elige un tema al azar segun las proporciones θ_d , (b) elige una palabra al azar segun ese tema.

El proceso generativo: vision intuitiva

Puente: de
variables
latentes a
texto

Introduccion:
datos no es-
tructurados

Análisis
textual:
Entender

Análisis
textual:
Estandarizar

Análisis
textual:
Describir

Análisis
textual:
Modelar con
LDA

LDA
Supervisado
(sLDA)

Mas alla de
BOW:
modelos de
lenguaje

Entrenamiento
con
gradiente
descendente

LDA **invierte** este proceso: observa las palabras y debe inferir los temas y proporciones.

El proceso generativo: esquema

Latent Dirichlet Allocation Intuition

Topics

gene	0.04
dna	0.02
genetic	0.01
...	

life	0.02
evolve	0.01
organism	0.01
...	

brain	0.04
neuron	0.02
nerve	0.01
...	

data	0.02
number	0.02
computer	0.01
...	

Documents

Seeking Life's Bare (Genetic) Necessities

COLD SPRING HARBOR, NEW YORK—How many **genes** does an organism need to **survive**? Last week at the genome meeting here, two genome researchers with radically different approaches presented complementary views of the basic genes needed for **life**. One research team, using **computer** analyses to compare known **genomes**, concluded that today's **organisms** can be sustained with just 250 genes, and that the earliest life forms required a mere 128 **genes**. The other researcher mapped genes in a simple parasite and estimated that for this organism, 800 genes are plenty to do the job—but that anything short of 100 wouldn't be enough. Although the numbers don't match precisely, those **predictions**

"are not all that far apart," especially in comparison to the 75,000 **genes** in the human genome, notes Steffen Andersson at the University in Sweden. "It may be that a concise answer may be more than just a **number**," she cautions. "There are more, and more **genes** are being sequenced. It's any newly **sequenced genome**," explains Arcady Mushegian, a computational molecular biologist at the National Center for Biotechnology Information (NCBI) in Bethesda, Maryland. Comparing an

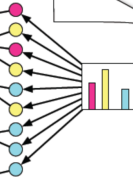


* Genome Mapping and Sequencing. Cold Spring Harbor, New York, May 8 to 12.

Stripping down. Computer analysis yields an estimate of the minimum modern and ancient genomes.

SCIENCE • VOL. 272 • 24 MAY 1996

Topic proportions and
assignments



Cada palabra w_{dn} se genera muestreando primero un tema z_{dn} de la mezcla del documento θ_d , y luego una palabra de la distribución $\beta_{z_{dn}}$ de ese tema.

El proceso generativo: las dos urnas

Puente: de
variables
latentes a
texto

Introducción:
datos no es-
tructurados

Análisis
textual:
Entender

Análisis
textual:
Estandarizar

Análisis
textual:
Describir

Análisis
textual:
Modelar con
LDA

LDA
Supervisado
(sLDA)

Más allá de
BOW:
modelos de
lenguaje

Entrenamiento
con
gradiente
descendente

Podemos visualizarlo con dos urnas:

Urna 1 por documento (θ_d): contiene bolitas etiquetadas con el nombre de cada tema. Las proporciones varían por documento. Saco una bolita: obtengo el tema z para esta palabra.

Urna 2 por tema (β_k): contiene bolitas etiquetadas con palabras del vocabulario. Proporciones fijas para todos los documentos. Saco una bolita: obtengo la palabra w .

Cada palabra del documento es el resultado de: sacar de la Urna 1 $\rightarrow$ identificar la Urna 2 correcta $\rightarrow$ sacar de esa Urna 2.

El proceso generativo: formalmente

Con K temas, vocabulario V , D documentos:

Para cada tema $k = 1, \dots, K$:

$$\beta_k \sim \text{Dirichlet}(\eta) \in \Delta^{V-1}$$

Vector de probabilidades sobre las V palabras del vocabulario.

Para cada documento $d = 1, \dots, D$:

$$\theta_d \sim \text{Dirichlet}(\alpha) \in \Delta^{K-1}$$

Vector de proporciones sobre los K temas.

Para cada palabra $n = 1, \dots, N_d$ del documento d :

$$z_{dn} \sim \text{Multinomial}(\theta_d)$$

$$w_{dn} \sim \text{Multinomial}(\beta_{z_{dn}})$$

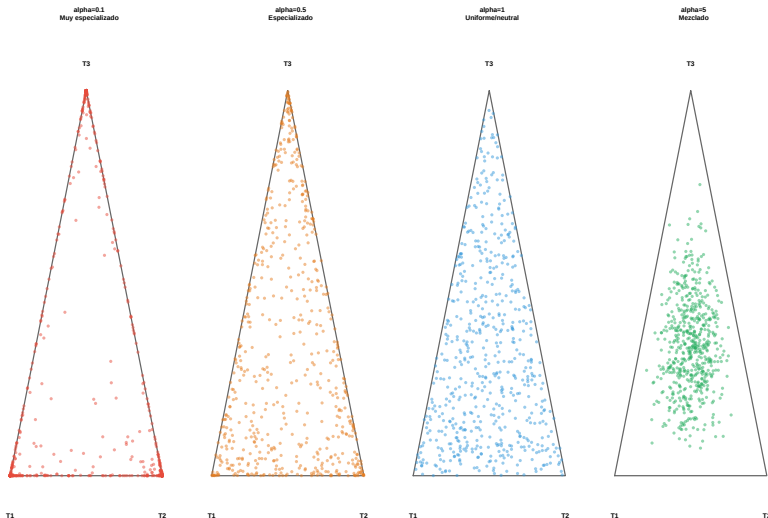
Que es la distribucion Dirichlet?

La **distribucion Dirichlet** genera vectores de probabilidad: vectores con entradas positivas que suman a 1.

- Cada muestra es un punto en el **simplex** Δ^{K-1} : el espacio de mezclas validas de K temas.
- Para $K = 3$ temas, el simplex es un triangulo equilatero: los vertices representan especializacion total en un tema y el centro representa mezcla equilibrada de los tres.

El parametro α controla la **concentracion** de las muestras dentro de ese triangulo.

La Dirichlet en el simplex: α simétrico



Con α simétrico, los documentos se concentran de forma uniforme hacia todos los vértices a la vez.

Interpretación del simplex

Cada punto dentro del triángulo es un documento θ_d :

- **Vertices** ($\alpha \ll 1$): documentos con un solo tema dominante. Típico en artículos científicos o noticias sectoriales.
- **Aristas** ($\alpha \approx 0.3-0.5$): documentos que mezclan principalmente dos de los tres temas.
- **Centro** ($\alpha \gg 1$): documentos que reparten uniformemente entre todos los temas.
- **Asimétrico** $\alpha = (\alpha_1, \dots, \alpha_K)$: permite que unos temas sean inherentemente más frecuentes que otros en toda la colección.

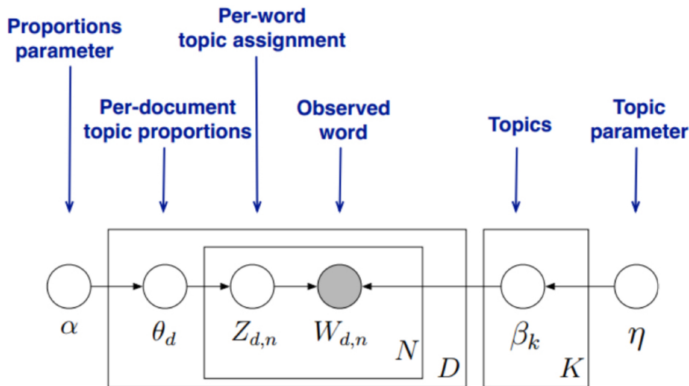
Elección práctica de α

Por defecto, `topicmodels::LDA()` usa $\alpha = 50/K$. Para noticias y artículos académicos, valores menores ($\alpha < 1$) dan temas más coherentes e interpretables.

Que captura cada parametro?

Par.	Distribucion	Significado
α	Dirichlet en θ_d	Grande: todos los documentos mezclan todos los temas. Pequeño: cada documento se especializa en pocos temas.
η	Dirichlet en β_k	Grande: cada tema usa muchas palabras. Pequeño: cada tema tiene pocas palabras muy características.
K	Escalar entero	Numero de temas: hiperparametro elegido por el analista (como k en K-means o df en splines).

El modelo grafico LDA



Los nodos son variables aleatorias, los edges indican dependencia. Las variables dentro de las cajas (placas) se repiten para cada documento, cada palabra, cada tema.

Lo que no hace LDA

Antes de modelar, es importante entender las limitaciones:

- Los temas **se mantienen** a través de todos los documentos: el tema 3 siempre tiene la misma distribución sobre palabras.
- Lo que **varia por documento** es la proporción de cada tema.
- El **orden de las palabras no importa**: LDA es un modelo *Bag of Words* (BOW). La frase “el estudiante pasó el examen” y “el examen pasó al estudiante” son idénticas para LDA.

Por esto, **LDA no generaría texto coherente**. Su objetivo no es generar texto sino descubrir temas.

La inferencia: EM aplicado a texto

Observamos las palabras $\{w_{dn}\}$; son latentes los temas $\{z_{dn}\}$ y las proporciones $\{\theta_d\}$.

La log-verosimilitud observada involucra una suma bajo el log (igual que en la mixtura gaussiana): no tiene solución cerrada.

Se aplica **EM variacional** o **muestreo de Gibbs**:

- **Paso E:** estimar $P(z_{dn} = k \mid w_{dn}, \theta_d, \beta_k)$, la probabilidad posterior de que la palabra w_{dn} provenga del tema k . Análogo a la responsabilidad $\hat{\gamma}_i$.
- **Paso M:** actualizar θ_d (proporciones del documento) y β_k (distribución de palabras por tema) dado el Paso E.

Corpus de ejemplo: Associated Press

Puente: de
variables
latentes a
texto

Introducción:
datos no es-
tructurados

Análisis
textual:
Entender

Análisis
textual:
Estandarizar

Análisis
textual:
Describir

Análisis
textual:
Modelar con
LDA

LDA
Supervisado
(sLDA)

Más allá de
BOW:
modelos de
lenguaje

Entrenamiento
con
gradiente
descendente

Para LDA usamos el corpus **AssociatedPress**: 2246 artículos de noticias de la agencia AP (1988). Ya viene como DTM en el paquete `topicmodels`.

```
library(topicmodels)
data("AssociatedPress", package = "topicmodels")
AssociatedPress
```

```
## <<DocumentTermMatrix (documents: 2246, terms: 10473)>>
## Non-/sparse entries: 302031/23220327
## Sparsity           : 99%
## Maximal term length: 18
## Weighting           : term frequency (tf)
```

A diferencia de un solo libro, este corpus tiene artículos de política, deportes, economía e internacional: esperamos **temas bien separados y fáciles de interpretar**.

Ajuste del modelo LDA. Preparación de datos

```
library(tidytext)
library(tidyr)
library(SnowballC)

# -----
# 1. Convertir la DTM original a formato tidy
ap_tidy <- tidy(AssociatedPress)

# 2. Limpieza básica: stopwords, números, longitud mínima
ap_tidy <- ap_tidy %>%
  anti_join(stop_words, by = c("term" = "word")) %>% # stopwords estándar
  filter(!grepl("[^a-z]", term)) %>% # solo letras (quita números y símbolos)
  filter(nchar(term) >= 3) # al menos 3 caracteres

# 3. Filtro por categoría gramatical (POS) con el diccionario Moby
data("parts_of_speech", package = "tidytext") # dataset interno de tidytext

# Mantenemos solo palabras con categoría de contenido:
# Noun, Verb (todas las formas), Adjective
pos_ok <- parts_of_speech %>%
  filter(grepl("Noun|Verb|Adjective", pos))

ap_tidy <- ap_tidy %>%
  semi_join(pos_ok, by = c("term" = "word"))

# 4. Stemming: reducimos cada término a su raíz para unificar variantes
ap_tidy <- ap_tidy %>%
  mutate(stem = SnowballC::wordStem(term)) %>%
  group_by(document, stem) %>%
  summarise(count = sum(count), .groups = "drop") %>%
  rename(term = stem)
```

Ajuste del modelo LDA. Preparación de datos

Puente: de
variables
latentes a
texto

Introducción:
datos no es-
tructurados

Análisis
textual:
Entender

Análisis
textual:
Estandarizar

Análisis
textual:
Describir

Análisis
textual:
Modelar con
LDA

LDA
Supervisado
(sLDA)

Más allá de
BOW:
modelos de
lenguaje

Entrenamiento
con
gradiente
descendente

```
# 5. Filtro final por frecuencia de documento (document frequency)
doc_stats <- ap_tidy %>%
  group_by(term) %>%
  summarise(doc_freq = n(), .groups = "drop")
n_docs_total <- n_distinct(ap_tidy$document)
```

```
ap_tidy <- ap_tidy %>%
  inner_join(doc_stats, by = "term") %>%
  filter(doc_freq >= 5, doc_freq <= 0.5 * n_docs_total) %>%
  dplyr::select(-doc_freq)
```

```
# 6. Reconstruir la DTM limpia
ap_dtm_clean <- ap_tidy %>%
  cast_dtm(document, term, count)
```

```
# Verificar dimensiones
ap_dtm_clean
```

```
## <<DocumentTermMatrix (documents: 2246, terms: 4681)>>
## Non-/sparse entries: 182353/10331173
## Sparsity : 98%
## Maximal term length: 14
## Weighting : term frequency (tf)
```

Ajuste del modelo LDA

```
ap_lda <- LDA(ap_dtm_clean, k = 8,  
              method = "Gibbs",  
              control = list(seed = 12345, iter = 3000, burnin = 1000,  
                             alpha = 0.1, delta = 0.1)) # delta es eta  
  
ap_topics <- tidy(ap_lda, matrix = "beta")  
top_terms <- ap_topics %>%  
  group_by(topic) %>%  
  slice_max(beta, n = 8) %>%  
  ungroup() %>%  
  arrange(topic, -beta)
```

beta es la probabilidad de cada palabra dado el tema:

$$\beta_{kv} = P(w = v \mid \text{topic} = k).$$

Con $K = 8$ temas esperamos descubrir temas temáticos claramente diferenciados: política, salud, economía, internacional...

Palabras características de los 8 temas

Puente: de
variables
latentes a
texto

Introducción:
datos no es-
tructurados

Análisis
textual:
Entender

Análisis
textual:
Estandarizar

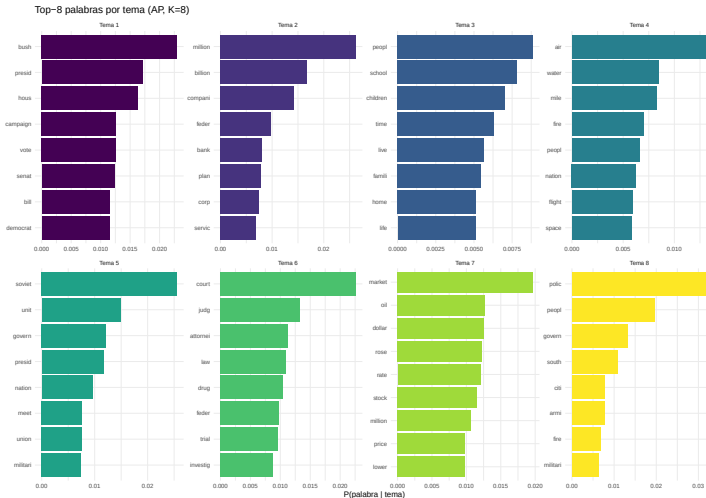
Análisis
textual:
Describir

Análisis
textual:
Modelar con
LDA

LDA
Supervisado
(sLDA)

Más allá de
BOW:
modelos de
lenguaje

Entrenamiento
con
gradiente
descendente



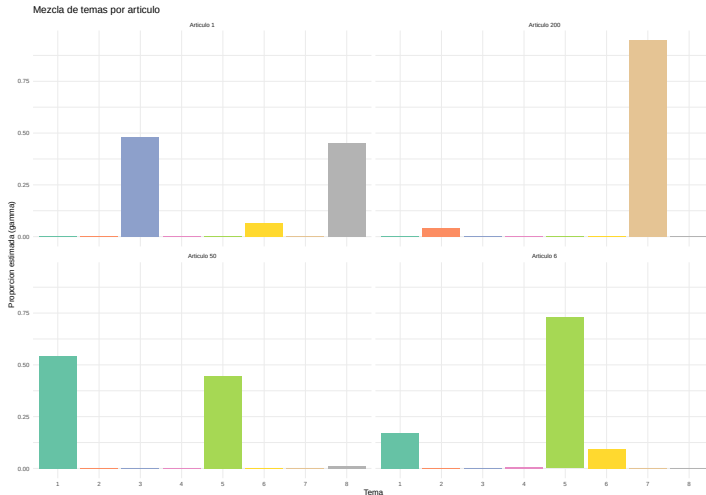
Interpretación de los temas

Los temas descubiertos por LDA en las noticias AP son interpretables:

LDA funciona bien aquí porque...

El corpus tiene **muchos documentos cortos** con temas bien delimitados. Con α pequeño, cada artículo tiende a pertenecer mayoritariamente a un solo tema.

Proporciones de temas por artículo



Puente: de
variables
latentes a
texto

Introduccion:
datos no es-
tructurados

Analisis
textual:
Entender

Analisis
textual:
Estandarizar

Analisis
textual:
Describir

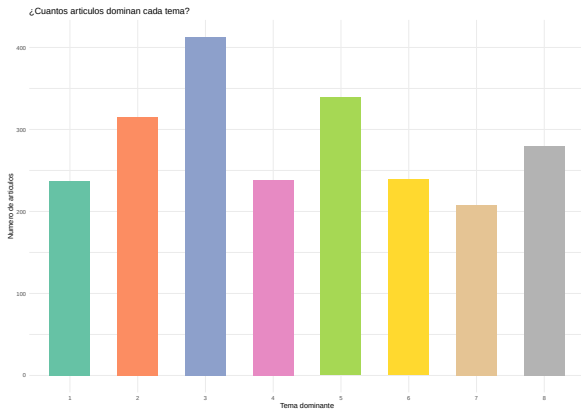
Analisis
textual:
Modelar con
LDA

LDA
Supervisado
(sLDA)

Mas alla de
BOW:
modelos de
lenguaje

Entrenamiento
con
gradiente
descendente

Distribucion global de temas dominantes



Esto permite una clasificacion blanda: cada articulo puede pertenecer a varios temas con distinta intensidad.

Puente: de
variables
latentes a
texto

Introducción:
datos no es-
tructurados

Análisis
textual:
Entender

Análisis
textual:
Estandarizar

Análisis
textual:
Describir

Análisis
textual:
Modelar con
LDA

**LDA
Supervisado
(sLDA)**

Más allá de
BOW:
modelos de
lenguaje

Entrenamiento
con
gradiente
descendente

LDA Supervisado (sLDA)

La limitacion de LDA no supervisado

Puente: de
variables
latentes a
texto

Introduccion:
datos no es-
tructurados

Análisis
textual:
Entender

Análisis
textual:
Estandarizar

Análisis
textual:
Describir

Análisis
textual:
Modelar con
LDA

LDA
Supervisado
(sLDA)

Más allá de
BOW:
modelos de
lenguaje

Entrenamiento
con
gradiente
descendente

LDA descubre temas de forma **no supervisada**: los temas emergentes no están guiados por ninguna variable de interés.

- **Problema práctico:** suponga que tiene 10000 reseñas de productos con calificación de 1 a 5 estrellas.
- LDA podría descubrir temas sobre “embalaje”, “precio” y “servicio al cliente”. Pero no hay garantía de que esos temas sean los más relevantes para **predecir la calificación**.
- **Solución:** el **LDA Supervisado (sLDA)** añade una variable respuesta y_d vinculada a las proporciones de temas θ_d de cada documento.

El modelo sLDA: proceso generativo

Puente: de
variables
latentes a
texto

Introducción:
datos no es-
tructurados

Análisis
textual:
Entender

Análisis
textual:
Estandarizar

Análisis
textual:
Describir

Análisis
textual:
Modelar con
LDA

LDA
Supervisado
(sLDA)

Más allá de
BOW:
modelos de
lenguaje

Entrenamiento
con
gradiente
descendente

sLDA extiende LDA con un componente predictivo:

Igual que en LDA:

$$\theta_d \sim \text{Dirichlet}(\alpha), \quad z_{dn} \sim \text{Multinomial}(\theta_d), \quad w_{dn} \sim \text{Multinomial}(\beta_{z_{dn}})$$

Adicionalmente, la respuesta del documento:

$$y_d \mid z_{d1}, \dots, z_{dN_d}, \eta, \sigma^2 \sim N(\eta^T \bar{z}_d, \sigma^2)$$

donde $\bar{z}_d = \frac{1}{N_d} \sum_n z_{dn}$ es la proporción empírica de temas en el documento d , y $\eta \in \mathbb{R}^K$ son los coeficientes que miden el efecto de cada tema sobre la respuesta.

sLDA: interpretacion de los coeficientes

Puente: de
variables
latentes a
texto

Introduccion:
datos no es-
tructurados

Analisis
textual:
Entender

Analisis
textual:
Estandarizar

Analisis
textual:
Describir

Analisis
textual:
Modelar con
LDA

LDA
Supervisado
(sLDA)

Mas alla de
BOW:
modelos de
lenguaje

Entrenamiento
con
gradiente
descendente

El vector η conecta los temas con la variable respuesta:

- $\eta_k > 0$: documentos con alta proporcion del tema k tienden a tener y_d alto (por ejemplo, reseñas positivas).
- $\eta_k < 0$: documentos con alta proporcion del tema k tienden a tener y_d bajo (por ejemplo, reseñas negativas).
- $\eta_k \approx 0$: el tema k no es informativo para predecir la respuesta.

Resultado: los temas aprendidos son, por construccion, los mas relevantes para predecir y_d . Son variables latentes **guiadas** por la respuesta.

Regresión sobre LDA: principio ilustrado (NO es sLDA)

Puente: de
variables
latentes a
texto

Introducción:
datos no es-
tructurados

Análisis
textual:
Entender

Análisis
textual:
Estandarizar

Análisis
textual:
Describir

Análisis
textual:
Modelar con
LDA

LDA
Supervisado
(sLDA)

Más allá de
BOW:
modelos de
lenguaje

Entrenamiento
con
gradiente
descendente

```
# Paso 1: proporciones de temas por artículo
ap_doc_topics <- tidy(ap_lda, matrix = "gamma") %>%
  pivot_wider(names_from = topic,
               values_from = gamma,
               names_prefix = "t") %>%
  mutate(document = as.integer(document))

# Paso 2: longitud del artículo como variable respuesta proxy
ap_lengths <- tibble(
  document = seq_len(nrow(AssociatedPress)),
  longitud = rowSums(as.matrix(AssociatedPress)))

# Paso 3: regresión de la respuesta sobre proporciones de temas
slda_data <- ap_doc_topics %>%
  left_join(ap_lengths, by = "document")
lm_slda <- lm(longitud ~ t1 + t2 + t3 + t4 + t5 + t6 + t7 + t8, data = slda_data)
round(coef(lm_slda), 1)
```

## (Intercept)	t1	t2	t3	t4	t5
## 155.1	88.6	51.5	5.6	12.4	94.7
## t6	t7	t8			
## 59.8	-1.5	NA			

Limitaciones de regresión sobre LDA. 1

- **El LDA aprende los temas sin conocer la variable dependiente**
Los tópicos se extraen únicamente a partir de las palabras, ignorando la respuesta (longitud), por lo que no están pensados para explicarla.
- **La regresión se ajusta *después* de obtener los temas**
Primero se entrena el LDA y luego se usa la salida como predictor; no hay un aprendizaje simultáneo palabras+respuesta.
- **La respuesta no influye en la asignación de palabras a temas**
En sLDA genuino la variable dependiente guía la formación de los tópicos; aquí esa retroalimentación no existe.
- **No es un modelo generativo conjunto**
sLDA modela palabras y respuesta bajo una misma estructura latente; tu código es una concatenación de dos modelos separados.

Limitaciones de regresión sobre LDA. 2

Puente: de
variables
latentes a
texto

Introducción:
datos no es-
tructurados

Análisis
textual:
Entender

Análisis
textual:
Estandarizar

Análisis
textual:
Describir

Análisis
textual:
Modelar con
LDA

LDA
Supervisado
(sLDA)

Más allá de
BOW:
modelos de
lenguaje

Entrenamiento
con
gradiente
descendente

- **Las proporciones de los temas suman 1 y generan colinealidad perfecta**
Incluir todos los temas en una regresión con intercepto crea redundancia exacta, lo que obliga a excluir un tema manualmente.
- **Falta un mecanismo integrado de predicción**
El sLDA auténtico predice la respuesta de documentos nuevos de forma natural; aquí la predicción depende de dos pasos no optimizados conjuntamente.

Puente: de
variables
latentes a
texto

Introducción:
datos no es-
tructurados

Análisis
textual:
Entender

Análisis
textual:
Estandarizar

Análisis
textual:
Describir

Análisis
textual:
Modelar con
LDA

LDA
Supervisado
(sLDA)

Más allá de
BOW:
modelos de
lenguaje

Entrenamiento
con
gradiente
descendente

En una aplicación real se usaría:

- Calificación de estrellas en reseñas de productos.
- Popularidad de un artículo (clicks, shares).
- Evaluación de desempeño en retroalimentación de empleados.

El modelo garantiza que los temas encontrados sean los más relevantes para predecir esa variable.

Puente: de
variables
latentes a
texto

Introducción:
datos no es-
tructurados

Análisis
textual:
Entender

Análisis
textual:
Estandarizar

Análisis
textual:
Describir

Análisis
textual:
Modelar con
LDA

LDA
Supervisado
(sLDA)

**Mas alla de
BOW:
modelos de
lenguaje**

Entrenamiento
con
gradiente
descendente

Mas alla de BOW: modelos de lenguaje

La limitacion central de LDA

Puente: de
variables
latentes a
texto

Introduccion:
datos no es-
tructurados

Analisis
textual:
Entender

Analisis
textual:
Estandarizar

Analisis
textual:
Describir

Analisis
textual:
Modelar con
LDA

LDA
Supervisado
(sLDA)

Mas alla de
BOW:
modelos de
lenguaje

Entrenamiento
con
gradiente
descendente

LDA es un modelo **Bag of Words** el orden de las palabras no importa.

- “El estudiante paso el examen” y “El examen paso al estudiante” son identicos para LDA.
- Esto es una restriccion severa: el significado del lenguaje depende del contexto y del orden.

Pregunta: ¿como representar una palabra de forma que su significado dependa de las palabras que la rodean?

Puente: de
variables
latentes a
texto

Introducción:
datos no es-
tructurados

Análisis
textual:
Entender

Análisis
textual:
Estandarizar

Análisis
textual:
Describir

Análisis
textual:
Modelar con
LDA

LDA
Supervisado
(sLDA)

Más allá de
BOW:
modelos de
lenguaje

Entrenamiento
con
gradiente
descendente

El paso clave

Necesitamos pasar de contar palabras (LDA, BOW) a **representar palabras como vectores** en un espacio donde la cercanía semántica corresponde a cercanía geométrica.

De palabras a vectores: embeddings

Puente: de
variables
latentes a
texto

Introducción:
datos no es-
tructurados

Análisis
textual:
Entender

Análisis
textual:
Estandarizar

Análisis
textual:
Describir

Análisis
textual:
Modelar con
LDA

LDA
Supervisado
(sLDA)

Más allá de
BOW:
modelos de
lenguaje

Entrenamiento
con
gradiente
descendente

Un **embedding** es una función $\phi : \text{Vocabulario} \rightarrow \mathbb{R}^d$ que asigna a cada palabra un vector denso.

Dos palabras semanticamente similares deben tener vectores cercanos:

$$\phi(\text{"rey"}) - \phi(\text{"hombre"}) + \phi(\text{"mujer"}) \approx \phi(\text{"reina"})$$

Esta es la misma idea del kernel via embedding de la Sesión 4:

$K(x, x') = \langle \phi(x), \phi(x') \rangle$, pero ahora ϕ es **aprendido desde los datos**, no diseñado a mano.

Word2Vec fue uno de los primeros métodos populares: aprende ϕ predeciendo palabras vecinas desde el contexto.

La hipótesis distribucional

La base teórica de los embeddings es la **hipótesis distribucional**:

“A word is known by the company it keeps” (Firth, 1957)

Palabras que aparecen en contextos similares tienen significados similares.

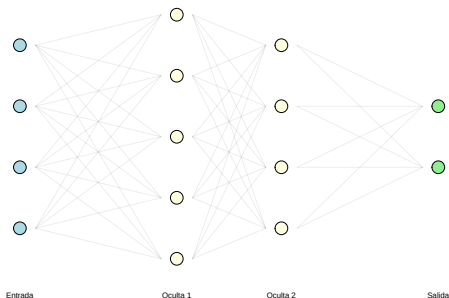
Ejemplos:

- “gato” y “perro” aparecen cerca de “come”, “duerme”, “peludo”, “mascota” $\Rightarrow$ sus vectores se acercan.
- “banco” (financiero) aparece cerca de “dinero”, “credito”, “prestamo” $\Rightarrow$ su vector apunta hacia finanzas.
- “banco” (peces) aparece cerca de “cardumen”, “agua”, “pesca” $\Rightarrow$ mismo vector, pero ahora jalado también hacia el dominio marino.

Word2Vec explota esta hipótesis a escala masiva.

Redes neuronales densas: punto de partida

Ustedes conocen una **red neuronal densa** (fully connected):



Entrada $x \rightarrow$ capas ocultas con funciones de activación no lineales $\rightarrow$ salida $\hat{y}$.
Cada capa aprende representaciones progresivamente abstractas.

Word2Vec: arquitectura simplificada

Word2Vec es una red densa muy simple con un objetivo específico: predecir la palabra central dado su contexto (CBOW) o predecir el contexto dado la palabra central (Skip-gram).

La clave: la red no tiene capas ocultas profundas. Solo:

$$\text{one-hot}(w) \xrightarrow{W} \text{embedding}(w) \xrightarrow{W'} \text{softmax}$$

Siempre hay **dos matrices**:

- $W \in \mathbb{R}^{|V| \times d}$
Cada fila $\mathbf{v}_i$ es un vector de dimensión d .
- $W' \in \mathbb{R}^{d \times |V|}$
Cada columna $\mathbf{u}_j$ es un vector de dimensión d .

Pero **qué representa cada una** depende de si usamos Skip-gram o CBOW.

Las dos matrices: ilustracion



Puente: de
variables
latentes a
texto

Introduccion:
datos no es-
tructurados

Analisis
textual:
Entender

Analisis
textual:
Estandarizar

Analisis
textual:
Describir

Analisis
textual:
Modelar con
LDA

LDA
Supervisado
(sLDA)

Mas alla de
BOW:
modelos de
lenguaje

Entrenamiento
con
gradiente
descendente

Corpus y vocabulario (ejemplo minimo)

Puente: de
variables
latentes a
texto

Introduccion:
datos no es-
tructurados

Análisis
textual:
Entender

Análisis
textual:
Estandarizar

Análisis
textual:
Describir

Análisis
textual:
Modelar con
LDA

LDA
Supervisado
(sLDA)

Mas alla de
BOW:
modelos de
lenguaje

Entrenamiento
con
gradiente
descendente

- Frase 1: **“el gato come pescado”**
- Frase 2: **“el perro come carne”**
- Vocabulario ($|V| = 6$): el, gato, come, pescado, perro, carne
- Ventana de contexto: 1 palabra a cada lado

Representacion one-hot:

Palabra	el	gato	come	pescado	perro	carne
el	1	0	0	0	0	0
gato	0	1	0	0	0	0
come	0	0	1	0	0	0
pescado	0	0	0	1	0	0
perro	0	0	0	0	1	0
carne	0	0	0	0	0	1

One-hot es una idea conceptual; la implementación real es:

- La palabra gato tiene el índice 2.
- La capa de entrada simplemente toma la fila 2 de W como la activación de la capa oculta.
- Eso es equivalente a multiplicar el one-hot por W pero mucho más eficiente.

CBOW: Predicción de la palabra central

Puente: de
variables
latentes a
texto

Introducción:
datos no es-
tructurados

Análisis
textual:
Entender

Análisis
textual:
Estandarizar

Análisis
textual:
Describir

Análisis
textual:
Modelar con
LDA

LDA
Supervisado
(sLDA)

Más allá de
BOW:
modelos de
lenguaje

Entrenamiento
con
gradiente
descendente

Corpus: “*el gato come pescado*” (frase 1)

Objetivo: predecir la palabra centro “**come**” usando su contexto inmediato (ventana = 1):

- Contexto izquierdo: gato
- Contexto derecho: pescado

1. Entrada one-hot ($|V|=6$, orden: el, gato, come, pescado, perro, carne)

$$\text{one-hot(gato)} = [0, 1, 0, 0, 0, 0], \quad \text{one-hot(pescado)} = [0, 0, 0, 1, 0, 0]$$

El vector de entrada a la red es el promedio (o suma) de los one-hot del contexto:

$$\mathbf{x} = \frac{1}{2} ([0, 1, 0, 0, 0, 0] + [0, 0, 0, 1, 0, 0]) = [0, 0.5, 0, 0.5, 0, 0]$$

2. Capa oculta (embedding del contexto, $d = 2$)

$$\mathbf{h} = \frac{1}{|\mathcal{C}|} \sum_{p \in \mathcal{C}} \mathbf{v}_p$$

donde $\mathcal{C}$ es el conjunto de palabras de contexto (aquí gato y pescado) y $\mathbf{v}_p$ es la fila de W correspondiente a la palabra p . En nuestro ejemplo:

$$\mathbf{h} = \frac{\mathbf{v}_{\text{gato}} + \mathbf{v}_{\text{pescado}}}{2}$$

3. Capa de salida (softmax)

Para cada posible palabra centro j calculamos un puntaje:

$$z_j = \mathbf{u}_j \cdot \mathbf{h}$$

La probabilidad de que la palabra centro sea “come” es:

$$P(\text{come} \mid \text{gato, pescado}) = \frac{\exp(\mathbf{u}_{\text{come}} \cdot \mathbf{h})}{\sum_{j=1}^V \exp(\mathbf{u}_j \cdot \mathbf{h})}$$

4. Entrenamiento

- **Salida deseada:** one-hot de “come” = $[0, 0, 1, 0, 0, 0]$
- La **pérdida de entropía cruzada** compara esta distribución objetivo con la distribución predicha.
- El gradiente **empuja** a que el producto punto $\mathbf{u}_{\text{come}} \cdot \mathbf{h}$ sea grande, mientras que los demás $\mathbf{u}_k \cdot \mathbf{h}$ sean pequeños.

En resumen: CBOW usa el contexto para predecir el centro.

- La capa oculta $\mathbf{h}$ es el promedio de los vectores de entrada $\mathbf{v}$ (filas de W) de las palabras de contexto.
- El modelo aprende vectores de salida $\mathbf{u}$ (columnas de W') que representan a cada palabra **como centro**.
- **Los embeddings finales (palabra como centro) son los vectores $\mathbf{u}$, no los $\mathbf{v}$.**

Además, al compartir contextos, palabras como *gato* y *perro* acaban con vectores de entrada $\mathbf{v}$ similares (útiles para representar su comportamiento contextual), pero es el vector $\mathbf{u}$ el que captura su significado como palabra central.

CBOW: ¿por qué palabras similares tienen vectores parecidos?

Puente: de
variables
atentes a
texto

Introducción:
datos no es-
tructurados

Análisis
textual:
Entender

Análisis
textual:
Estandarizar

Análisis
textual:
Describir

Análisis
textual:
Modelar con
LDA

LDA
Supervisado
(sLDA)

Más allá de
BOW:
modelos de
lenguaje

Entrenamiento
con
gradiente
descendente

Corpus: “el gato come pescado” y “el perro come carne”.

En CBOW, el contexto es la entrada (promedio de $\mathbf{v}$) y la palabra central es la salida ($\mathbf{u}$).

- Para predecir **gato** y **perro** como centro, el modelo usa contextos similares (ej. **el** y **come**).
- La capa oculta es el promedio de los vectores de contexto:

$$\mathbf{h} = \frac{\mathbf{v}_{\text{el}} + \mathbf{v}_{\text{come}}}{2}$$

- El modelo maximiza:

$$\mathbf{u}_{\text{gato}} \cdot \mathbf{h} \quad \text{y} \quad \mathbf{u}_{\text{perro}} \cdot \mathbf{h}$$

- Para que ambos productos sean altos con el mismo $\mathbf{h}$, $\mathbf{u}_{\text{gato}}$ y $\mathbf{u}_{\text{perro}}$ deben apuntar en **direcciones similares**.
- Análogamente, $\mathbf{v}_{\text{pescado}}$ y $\mathbf{v}_{\text{carne}}$ se alinean porque ambos aparecen como contexto de los mismos centros.

Con millones de ejemplos, emergen agrupaciones:

Vecindad en el espacio

Causa

u_{gato} $\approx$ **u**_{perro}

v_{pescado} $\approx$ **v**_{carne}

u_{rey} $\approx$ **u**_{reina}

u_{París} $\approx$ **u**_{Madrid}

Comparten
contextos (son
centros similares)
Aparecen como
contexto de los
mismos centros
Contextos de
realiza y poder
Contextos de
capitales
europeas

Skip-gram: Predicción del contexto

Puente: de
variables
atentes a
texto

Introduccion:
datos no es-
tructurados

Analisis
textual:
Entender

Analisis
textual:
Estandarizar

Analisis
textual:
Describir

Analisis
textual:
Modelar con
LDA

LDA
Supervisado
(sLDA)

Mas alla de
BOW:
modelos de
lenguaje

Entrenamiento
con
gradiente
descendente

Corpus: “*el gato come **pescado***” (frase 1)

Objetivo: predecir las palabras de contexto a partir de la palabra central
“**come**” (ventana = 1).

- Contexto a predecir (dos ejemplos independientes):
 - izquierdo: gato
 - derecho: pescado

1. Entrada one-hot ($|V|=6$, orden: el, gato, come, pescado, perro, carne)

$$\text{one-hot}(\text{come}) = [0, 0, 1, 0, 0, 0]$$

El vector de entrada es el one-hot de la palabra central “come”.

2. Capa oculta (embedding de la palabra centro, $d = 2$)

La capa oculta es exactamente el vector de entrada de “come” (fila de W):

$$\mathbf{h} = \mathbf{v}_{\text{come}}$$

Este vector $\mathbf{h}$ representa la **información de la palabra central**.

3. Capa de salida (softmax)

Para cada posible palabra de contexto j calculamos un puntaje:

$$z_j = \mathbf{u}_j \cdot \mathbf{h}$$

La probabilidad de que una palabra de contexto o aparezca dado el centro “come” es:

$$P(o \mid \text{come}) = \frac{\exp(\mathbf{u}_o \cdot \mathbf{v}_{\text{come}})}{\sum_{k=1}^V \exp(\mathbf{u}_k \cdot \mathbf{v}_{\text{come}})}$$

4. Entrenamiento

- **Salidas deseadas:** one-hot de cada contexto real:
 - para gato: $[0, 1, 0, 0, 0, 0]$
 - para pescado: $[0, 0, 0, 1, 0, 0]$
- La **pérdida de entropía cruzada** se calcula por separado para cada par (centro, contexto).
- El gradiente **aumenta** $\mathbf{u}_{\text{gato}} \cdot \mathbf{v}_{\text{come}}$ y $\mathbf{u}_{\text{pescado}} \cdot \mathbf{v}_{\text{come}}$, y **reduce** esos productos para las demás palabras.

En resumen: Skip-gram usa el centro para predecir el contexto.

- La capa oculta es el vector de entrada **v** de la palabra central (fila de W).
- El modelo aprende vectores de salida **u** (columnas de W') que representan a cada palabra **como contexto**.
- **Los embeddings finales (palabra como centro) son los vectores **v**, no los **u**.**

Además, palabras como *gato* y *perro*, que comparten contextos, acaban con vectores **v** similares, y palabras como *pescado* y *carne* ven alineados sus vectores **u**.

Skip-gram: ¿por qué palabras similares tienen vectores parecidos?

Puente: de
variables
latentes a
texto

Introducción:
datos no es-
tructurados

Análisis
textual:
Entender

Análisis
textual:
Estandarizar

Análisis
textual:
Describir

Análisis
textual:
Modelar con
LDA

LDA
Supervisado
(sLDA)

Más allá de
BOW:
modelos de
lenguaje

Entrenamiento
con
gradiente
descendente

Corpus: “el gato come pescado” y “el perro come carne”.

En Skip-gram, la palabra central es la entrada (**v**) y el contexto es la salida (**u**).

- **gato** y **perro** comparten el mismo contexto **come**.
- El modelo maximiza:

$$\mathbf{v}_{\text{gato}} \cdot \mathbf{u}_{\text{come}} \quad \text{y} \quad \mathbf{v}_{\text{perro}} \cdot \mathbf{u}_{\text{come}}$$

- Para que ambos productos punto sean altos con el mismo **u_{come}**, **v_{gato}** y **v_{perro}** deben apuntar en **direcciones similares**.
- De igual modo, **u_{pescado}** y **u_{carne}** se alinean porque comparten los mismos centros (gato, perro).

Con millones de ejemplos, emergen agrupaciones:

Vecindad en el espacio

$V_{\text{gato}} \approx V_{\text{perro}}$

$U_{\text{pescado}} \approx U_{\text{carne}}$

$V_{\text{rey}} \approx V_{\text{reina}}$

$V_{\text{París}} \approx V_{\text{Madrid}}$

Causa

Contextos de
animales
domésticos
Contextos de
alimentos
Contextos de
realiza y poder
Contextos de
capitales
europeas

Resumen de matrices

Modelo	W (filas v)	W' (columnas u)	Emb. F
Skip	Centro (entrada)	Contexto (salida)	v
CBOW	Contexto (entrada, promediado)	Centro (salida)	u

- Skip-gram: v_c representa la palabra como centro, u_o como contexto.
- CBOW: v_p representa las palabras como contexto, u_c como centro.

Conclusión: no hay una matriz que sea exclusivamente “de objetivo” o “de contexto”; el papel lo define la tarea de predicción.

Analogías vectoriales: ejemplos

La geometría del espacio de embeddings captura relaciones:

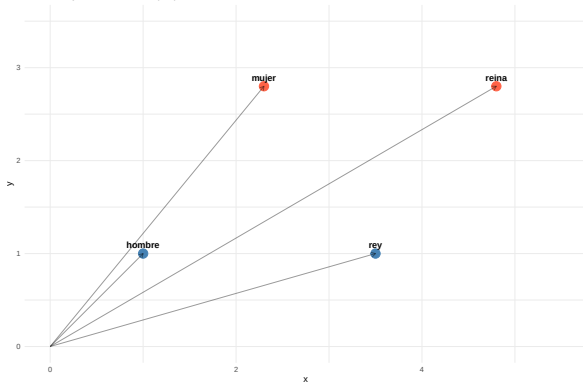
$$v_{\text{rey}} - v_{\text{hombre}} + v_{\text{mujer}} \approx v_{\text{reina}}$$

Analogia	Operacion vectorial
Rey : Reina = Hombre : ?	$v_{\text{rey}} - v_{\text{hombre}} + v_{\text{mujer}} \approx v_{\text{reina}}$
Bogota : Colombia = Lima : ?	$v_{\text{Bogota}} - v_{\text{Colombia}} + v_{\text{Peru}} \approx v_{\text{Lima}}$
Perro : Ladrar = Gato : ?	$v_{\text{perro}} - v_{\text{ladrar}} + v_{\text{maullar}} \approx v_{\text{gato}}$
Francia : Frances = Colombia : ?	$v_{\text{Francia}} - v_{\text{frances}} + v_{\text{espanol}} \approx v_{\text{Colombia}}$

Estas relaciones **emergen del entrenamiento**, no están programadas. Son el resultado de la estructura estadística del lenguaje capturada por Word2Vec.

El vector de analogía

1. Puntos y vectores desde (0,0)



Puente: de
variables
latentes a
texto

Introducción:
datos no es-
tructurados

Análisis
textual:
Entender

Análisis
textual:
Estandarizar

Análisis
textual:
Describir

Análisis
textual:
Modelar con
LDA

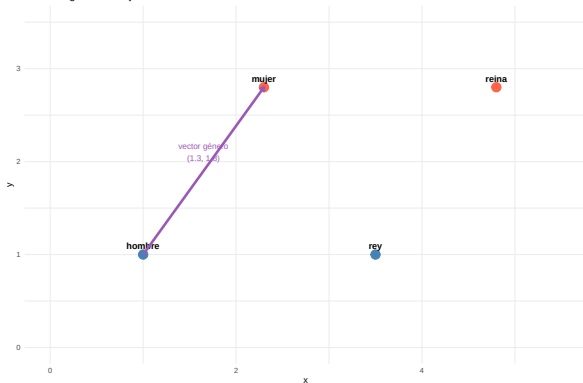
LDA
Supervisado
(sLDA)

Más allá de
BOW:
modelos de
lenguaje

Entrenamiento
con
gradiente
descendente

El vector de analogía

2. Vector género = mujer - hombre



Puente: de
variables
latentes a
texto

Introducción:
datos no es-
tructurados

Análisis
textual:
Entender

Análisis
textual:
Estandarizar

Análisis
textual:
Describir

Análisis
textual:
Modelar con
LDA

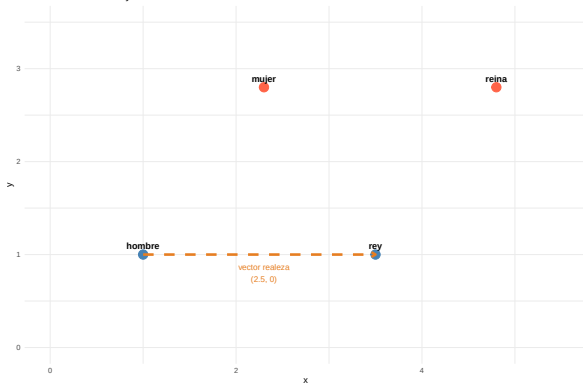
LDA
Supervisado
(sLDA)

Más allá de
BOW:
modelos de
lenguaje

Entrenamiento
con
gradiente
descendente

El vector de analogía

3. Vector realeza = rey - hombre



Puente: de
variables
latentes a
texto

Introducción:
datos no es-
tructurados

Análisis
textual:
Entender

Análisis
textual:
Estandarizar

Análisis
textual:
Describir

Análisis
textual:
Modelar con
LDA

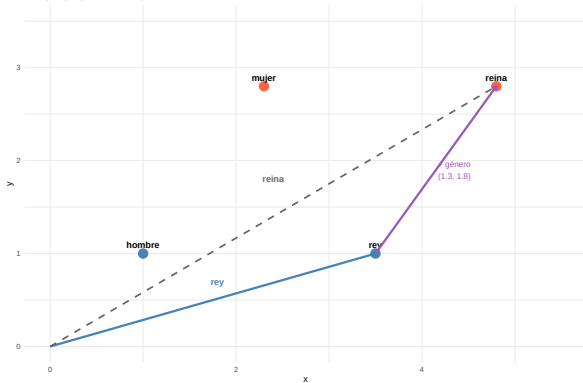
LDA
Supervisado
(sLDA)

Más allá de
BOW:
modelos de
lenguaje

Entrenamiento
con
gradiente
descendente

El vector de analogía

4. $\text{rey} + (\text{mujer} - \text{hombre}) = \text{reina}$



Puente: de
variables
latentes a
texto

Introducción:
datos no es-
tructurados

Análisis
textual:
Entender

Análisis
textual:
Estandarizar

Análisis
textual:
Describir

Análisis
textual:
Modelar con
LDA

LDA
Supervisado
(sLDA)

Más allá de
BOW:
modelos de
lenguaje

Entrenamiento
con
gradiente
descendente

Vecinos mas cercanos en Word2Vec (pre-entrenado)

La similitud coseno $\cos \theta = v_a \cdot v_b / (\|v_a\| \|v_b\|)$ mide cercanía semantica.
Resultados típicos en un modelo grande:

Consulta	Vecinos mas cercanos (sim. coseno)
"rey"	reina (0.89), principe (0.86), monarca (0.83)
"Paris"	Londres (0.91), Berlin (0.89), Madrid (0.87)
"banco"	credito (0.78), financiero (0.75), prestamo (0.73)
"gato"	perro (0.88), felino (0.85), mascota (0.82)
"bueno"	excelente (0.83), magnifico (0.80), genial (0.78)

Nota: "banco" agrupa vecinos **financieros** porque en un corpus de noticias ese contexto domina. Un corpus de biología marina daría vecinos completamente distintos.

Los embeddings son ESTATICOS

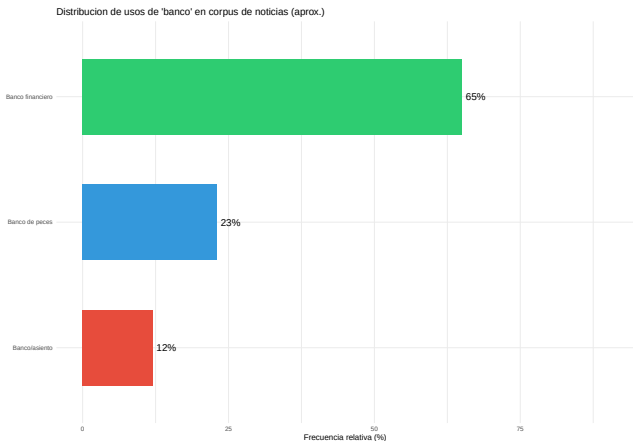
Aunque Word2Vec aprende **dos vectores** por palabra (fila de W y columna de W'), **ninguno** de ellos cambia según el contexto donde aparece la palabra. La palabra **“banco”** tiene exactamente los mismos vectores en:

Frase	Significado real	Vector W de “banco”
“Fui al <i>banco</i> a pagar el prestamo”	Institucion financiera	$[0.31, -0.12, \dots]$
“El <i>banco</i> de peces paso nadando”	Cardumen	$[0.31, -0.12, \dots]$
“Me sente en el <i>banco</i> del parque”	Asiento	$[0.31, -0.12, \dots]$

El vector final es una mezcla ponderada de todos los contextos en los que apareció “banco” durante el entrenamiento. Esto se llama **polisemia no resuelta**.

La limitación: polisemia no resuelta

El vector es una mezcla de todos sus significados ponderada por la frecuencia:



Puente: de
variables
latentes a
texto

Introduccion:
datos no es-
tructurados

Analisis
textual:
Entender

Analisis
textual:
Estandarizar

Analisis
textual:
Describir

Analisis
textual:
Modelar con
LDA

LDA
Supervisado
(sLDA)

Mas alla de
BOW:
modelos de
lenguaje

Entrenamiento
con
gradiente
descendente

Entrenamiento con gradiente descendente

¿Como aprende Word2Vec? El gradiente descendente

Puente: de variables latentes a texto

Introducción: datos no estructurados

Análisis textual: Entender

Análisis textual: Estandarizar

Análisis textual: Describir

Análisis textual: Modelar con LDA

LDA Supervisado (sLDA)

Más allá de BOW: modelos de lenguaje

Entrenamiento con gradiente descendente

Word2Vec (y todos los modelos neuronales) aprenden ajustando sus parámetros para **minimizar una función de pérdida** $\mathcal{L}$.

El mecanismo es el **descenso por gradiente**:

- 1 Evaluar la pérdida $\mathcal{L}(\theta)$ con los parámetros actuales.
- 2 Calcular el gradiente $\nabla_{\theta}\mathcal{L}$: la dirección en que la pérdida sube más rápido.
- 3 Moverse en la **dirección contraria** un paso pequeño η :

$$\theta \leftarrow \theta - \eta \nabla_{\theta}\mathcal{L}$$

- 4 Repetir hasta convergencia.

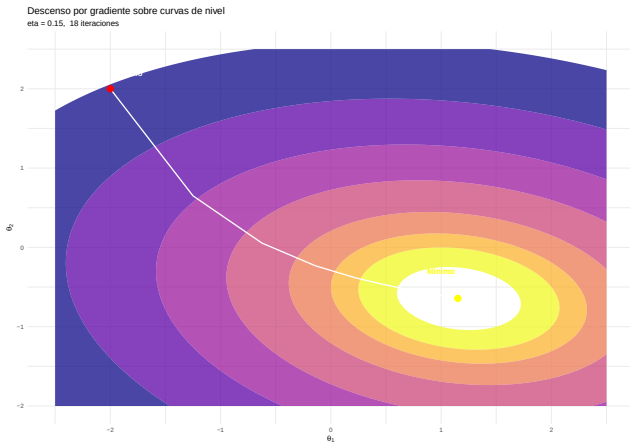
Analogía

Imagine que está en una montaña con niebla y quiere bajar al valle. En cada paso mide la pendiente local y da un paso hacia la dirección más empinada hacia abajo.

Un ejemplo trivariado: mínimos de una funcion

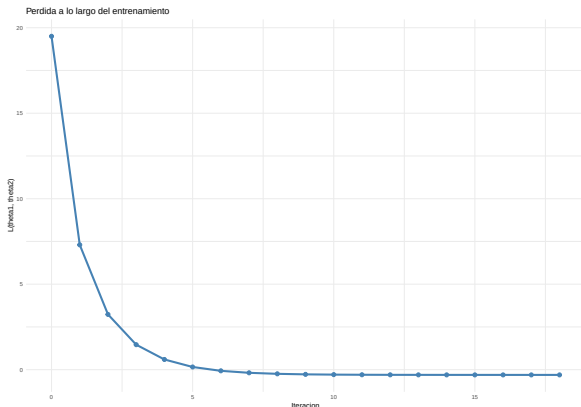
Usemos la funcion $\mathcal{L}(\theta_1, \theta_2) = (\theta_1 - 1)^2 + 2(\theta_2 + 0.5)^2 + 0.5\theta_1\theta_2$

que tiene un minimo unico. Partimos de $(\theta_1^{(0)}, \theta_2^{(0)}) = (-2, 2)$ y aplicamos gradiente descendente con $\eta = 0.15$.



Convergencia de la pérdida

Observamos como la pérdida disminuye en cada iteración:



El gradiente en cada punto indica la pendiente local. En cada paso, los parámetros se mueven un poco hacia el mínimo. Con una tasa de aprendizaje η adecuada, la pérdida converge.

Gradiente: ¿cómo se calcula?

Para la función de pérdida $\mathcal{L}(\theta_1, \theta_2) = (\theta_1 - 1)^2 + 2(\theta_2 + 0.5)^2 + 0.5 \theta_1 \theta_2$
las derivadas parciales son:

$$\frac{\partial \mathcal{L}}{\partial \theta_1} = 2(\theta_1 - 1) + 0.5 \theta_2$$

$$\frac{\partial \mathcal{L}}{\partial \theta_2} = 4(\theta_2 + 0.5) + 0.5 \theta_1$$

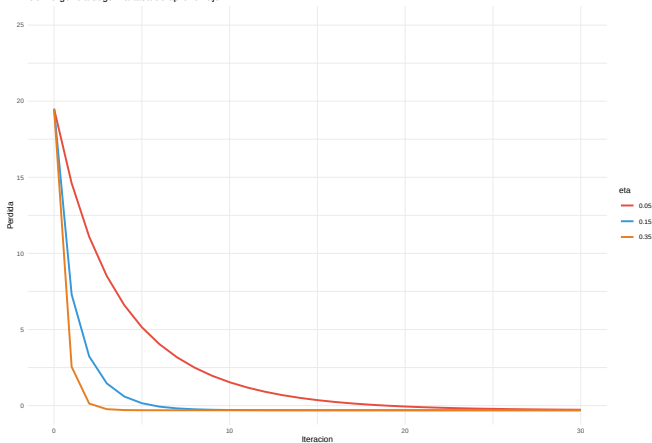
En cada iteración, evaluamos el gradiente en el punto actual y actualizamos
ambos parámetros **simultáneamente**:

$$\theta_1^{(t+1)} = \theta_1^{(t)} - \eta \left. \frac{\partial \mathcal{L}}{\partial \theta_1} \right|_{\theta^{(t)}}$$

$$\theta_2^{(t+1)} = \theta_2^{(t)} - \eta \left. \frac{\partial \mathcal{L}}{\partial \theta_2} \right|_{\theta^{(t)}}$$

El rol de la tasa de aprendizaje η

Convergencia segun la tasa de aprendizaje



- η pequeno: convergencia lenta pero segura.
- η adecuado: convergencia rapida.
- η grande: oscilacion o divergencia.

Puente: de variables latentes a texto

Introduccion: datos no estructurados

Analisis textual: Entender

Analisis textual: Estandarizar

Analisis textual: Describir

Analisis textual: Modelar con LDA

LDA Supervisado (sLDA)

Mas alla de BOW: modelos de lenguaje

Entrenamiento con gradiente descendente

Gradiente en Word2Vec: conexión

En Word2Vec, los “parámetros” son los vectores de embedding: $\theta = \{W, W'\}$ (millones de números).

La **perdida de entropía cruzada** compara la distribución predicha con la observada. El gradiente ajusta cada vector una pequeña cantidad en cada ejemplo de entrenamiento.

Con millones de frases, los vectores convergen a representaciones donde palabras similares quedan cerca.

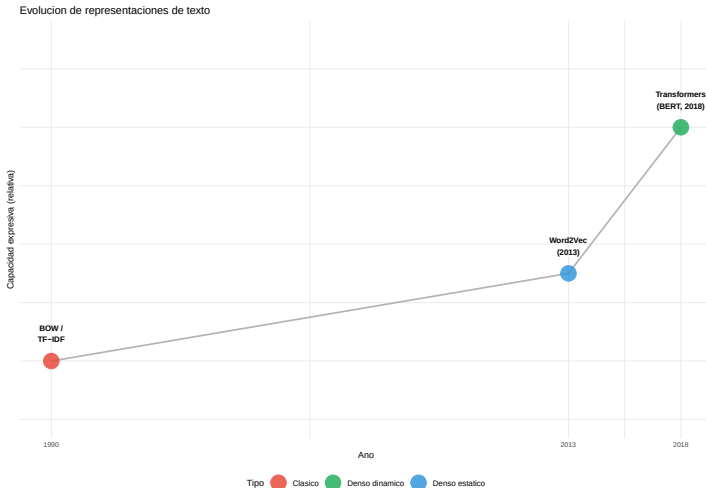
Gradiente en Word2Vec: conexión

Gradiente estocástico (SGD)

En la práctica no se usa una sola fórmula analítica: se toma un **mini-batch** de ejemplos, se estima el gradiente en ese subconjunto, y se actualiza. Esto es el **Stochastic Gradient Descent**.

Avance: de BOW a Transformers

El campo avanza en generaciones claramente distinguibles:



Resumen de la evolución

Modelo	Representación	Polisemia	Entrenamiento
BOW / TF-IDF	Dispersa, conteos	No	Conteo directo
Word2Vec	Densa, estática	No	SGD
Transformers (BERT)	Densa, dinámica	Si	SGD

El gran salto

De contar palabras (BOW/LDA) a representarlas como vectores (Word2Vec) a representarlas **según el contexto completo** (Transformers): cada generación resolvió la limitación clave de la anterior.

De Word2Vec a Transformers: el salto clave

Puente: de
variables
latentes a
texto

Introducción:
datos no es-
tructurados

Análisis
textual:
Entender

Análisis
textual:
Estandarizar

Análisis
textual:
Describir

Análisis
textual:
Modelar con
LDA

LDA
Supervisado
(sLDA)

Más allá de
BOW:
modelos de
lenguaje

Entrenamiento
con
gradiente
descendente

Word2Vec resolvió la representación densa y semántica, pero el vector de “banco” es el mismo en cualquier contexto.

Lo que Word2Vec resolvió

Representaciones **densas** y **semánticas** en lugar de one-hot. Permite medir similitud y hacer aritmética vectorial.

De Word2Vec a Transformers: el salto clave

Puente: de
variables
latentes a
texto

Introducción:
datos no es-
tructurados

Análisis
textual:
Entender

Análisis
textual:
Estandarizar

Análisis
textual:
Describir

Análisis
textual:
Modelar con
LDA

LDA
Supervisado
(sLDA)

Más allá de
BOW:
modelos de
lenguaje

Entrenamiento
con
gradiente
descendente

La limitación que los Transformers resolvieron

Word2Vec produce vectores **estáticos**: una palabra, un vector, sin importar el contexto. Los Transformers generan representaciones **dinámicas**: la misma palabra tiene un vector distinto según la frase completa donde aparece.

Para este curso: los Transformers son la arquitectura detrás de modelos como BERT, GPT y LLaMA. Su estudio profundo va más allá del alcance del curso, pero los usaremos como componente de BERTopic.

En R se acceden vía `reticulate` + HuggingFace; en Python, el paquete `transformers` es el estándar.

Puente: de
variables
latentes a
texto

Introduccion:
datos no es-
tructurados

Analisis
textual:
Entender

Analisis
textual:
Estandarizar

Analisis
textual:
Describir

Analisis
textual:
Modelar con
LDA

LDA
Supervisado
(sLDA)

Mas alla de
BOW:
modelos de
lenguaje

Entrenamiento
con
gradiente
descendente

BERTopic: modelado de temas con embeddings

LDA vs. BERTopic: la idea central

LDA representa documentos como **bolsas de palabras** (BOW) e infiere temas mediante EM. BERTopic toma un camino distinto:

- 1 Representa cada documento como un **vector denso** (embedding de Transformers) que captura su significado completo.
- 2 Reduce la dimension de esos vectores para poder agruparlos eficientemente.
- 3 Agrupa documentos similares con un algoritmo de **clustering**.
- 4 Extrae las palabras características de cada grupo con una version ponderada de TF-IDF.

La diferencia clave

LDA infiere temas como distribuciones probabilisticas sobre palabras. BERTopic **agrupa documentos** segun su significado semantico y luego etiqueta los grupos.

El pipeline de BERTopic paso a paso

Puente: de
variables
latentes a
texto

Introduccion:
datos no es-
tructurados

Análisis
textual:
Entender

Análisis
textual:
Estandarizar

Análisis
textual:
Describir

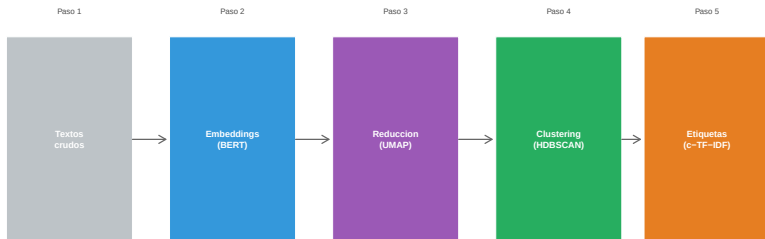
Análisis
textual:
Modelar con
LDA

LDA
Supervisado
(sLDA)

Mas alla de
BOW:
modelos de
lenguaje

Entrenamiento
con
gradiente
descendente

Pipeline de BERTopic



Paso 1: embeddings de documentos

Puente: de
variables
atentes a
texto

Introducción:
datos no es-
tructurados

Análisis
textual:
Entender

Análisis
textual:
Estandarizar

Análisis
textual:
Describir

Análisis
textual:
Modelar con
LDA

LDA
Supervisado
(sLDA)

Más allá de
BOW:
modelos de
lenguaje

Entrenamiento
con
gradiente
descendente

Cada documento se convierte en un vector en $\mathbb{R}^d$ (típicamente $d = 384$ o 768) usando un modelo de Transformers pre-entrenado como all-MiniLM-L6-v2.

La ventaja sobre BOW:

- El embedding captura el **significado semántico**: “el estudiante reprobo” y “el alumno no pasó la materia” tendrán vectores cercanos.
- No depende del vocabulario exacto ni del orden de las palabras.
- Maneja sinónimos, parafrasis y contexto naturalmente.

Conexión con lo visto

Es la misma idea del embedding de la Sesión 4 (kernel via feature map), pero ahora $\phi(d)$ proviene de una red de Transformers, no de un kernel diseñado a mano.

Paso 2: reduccion de dimensionalidad (UMAP)

Puente: de
variables
latentes a
texto

Introduccion:
datos no es-
tructurados

Análisis
textual:
Entender

Análisis
textual:
Estandarizar

Análisis
textual:
Describir

Análisis
textual:
Modelar con
LDA

LDA
Supervisado
(sLDA)

Mas alla de
BOW:
modelos de
lenguaje

Entrenamiento
con
gradiente
descendente

Los embeddings viven en un espacio de alta dimension ($d \approx 384$). El clustering directo en ese espacio falla por la **maldicion de la dimensionalidad**: todos los puntos quedan igualmente lejos entre si.

Se usa **UMAP** (Uniform Manifold Approximation and Projection) para reducir a $d' = 5$ o 10 dimensiones manteniendo la estructura local de vecindad.

Paso 2: Idea intuitiva de UMAP

Puente: de
variables
latentes a
texto

Introducción:
datos no es-
tructurados

Análisis
textual:
Entender

Análisis
textual:
Estandarizar

Análisis
textual:
Describir

Análisis
textual:
Modelar con
LDA

LDA
Supervisado
(sLDA)

Más allá de
BOW:
modelos de
lenguaje

Entrenamiento
con
gradiente
descendente

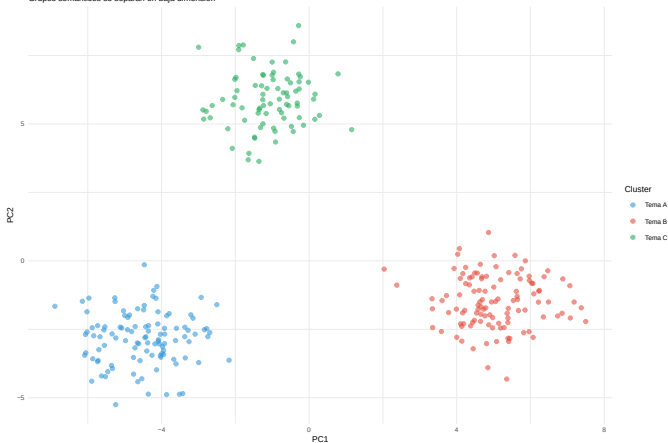
Construye un grafo de vecindad en el espacio original y luego aprende una proyección de baja dimensión que preserva ese grafo. Es similar en espíritu a PCA (Sesión 6), pero no lineal: puede capturar estructuras curvas.

Por qué UMAP y no PCA aquí?

PCA preserva varianza global (distancias grandes). UMAP preserva la **estructura local**: vecinos cercanos se mantienen cercanos, lo que facilita el clustering posterior.

UMAP: visualizacion del efecto

Proyeccion de embeddings en 2D (analógia a UMAP con PCA)
Grupos semanticos se separan en baja dimension



En la practica, UMAP produce separaciones aun mas nitidas porque preserva la topologia local del espacio semantico.

Paso 3: clustering con HDBSCAN

Puente: de
variables
latentes a
texto

Introducción:
datos no es-
tructurados

Análisis
textual:
Entender

Análisis
textual:
Estandarizar

Análisis
textual:
Describir

Análisis
textual:
Modelar con
LDA

LDA
Supervizado
(sLDA)

Más allá de
BOW:
modelos de
lenguaje

Entrenamiento
con
gradiente
descendente

Con los embeddings reducidos, BERTopic aplica **HDBSCAN** (Hierarchical Density-Based Spatial Clustering):

- Identifica grupos como **regiones densas** en el espacio.
- No requiere especificar K de antemano (a diferencia de K-means).
- Asigna documentos de baja densidad a la categoría **-1** (ruido).
- Produce clusters de tamaño y forma arbitrarios.

K-means / LDA	HDBSCAN
Requiere K prefijado	Infiere K automáticamente
Asigna todos los puntos	Puede marcar ruido
Clusters esféricos	Clusters de forma libre
Sensible a inicialización	Estable (jerárquico)

Paso 4: etiquetado con c-TF-IDF

Una vez formados los clusters, BERTopic los etiqueta con las palabras más características usando **c-TF-IDF** (class-based TF-IDF):

- Trata cada cluster como si fuera un solo documento gigante.
- Calcula TF-IDF entre clusters: una palabra es característica si es frecuente en ese cluster pero rara en los demás.

$$\text{c-TF-IDF}_{t,c} = \frac{f_{t,c}}{\sum_i f_{i,c}} \cdot \log\left(1 + \frac{A}{df_t}\right)$$

donde $f_{t,c}$ es la frecuencia del término t en la clase c y A es el número promedio de palabras por clase.

Resultado

Cada tema queda caracterizado por sus top- n palabras, igual que en LDA, pero los temas se formaron sobre significado semántico, no solo co-ocurrencia.

BERTopic en Python (paso a paso)

```
# Instalacion: pip install bertopic
from bertopic import BERTopic
from sentence_transformers import SentenceTransformer

# Corpus: lista de strings
docs = ["El estudiante reprobó el examen...",
        "La economía colombiana creció un 3%...",
        ...] # miles de documentos

# Paso 1: modelo de embeddings (BERT liviano)
embedding_model = SentenceTransformer("all-MiniLM-L6-v2")

# Pasos 2-4 encapsulados en BERTopic
topic_model = BERTopic(embedding_model = embedding_model,
                        language       = "spanish",
                        min_topic_size = 15,
                        nr_topics      = "auto")

# Ajuste
topics, probs = topic_model.fit_transform(docs)

# Top palabras por tema
topic_model.get_topic_info()
```

BERTopic: ventajas sobre LDA

Aspecto	LDA	BERTopic
Representación	BOW	Embedding semántico
Sinónimos	No	Si (por embedding)
Número de temas	Fijo (K)	Inferido automáticamente
Documentos cortos	Problemático	Robusto
Polisemia	No	Parcialmente (BERT)
Interpretabilidad	Alta (prob.)	Media (c-TF-IDF)
Recursos	Bajos	Moderados/Altos
Marco probabilístico	Si (Bayesiano)	No

BERTopic: desventajas y limitaciones

No todo es ventaja

BERTopic resuelve varios problemas de LDA pero introduce nuevos:

Desventajas principales:

- **No es un modelo probabilístico:** no hay una distribución sobre temas por documento como en LDA. La asignación es “dura” (un documento, un tema dominante).
- **Requiere mas recursos:** calcular embeddings de BERT para miles de documentos es computacionalmente costoso.
- **Menos interpretable estadísticamente:** no hay α , β ni posteriors. La incertidumbre no está cuantificada.
- **Dependiente de UMAP:** la calidad de los clusters depende de los hiperparámetros de reducción de dimensión.
- **Ruido (-1):** documentos “atípicos” se descartan, lo que puede ser una pérdida de información.
- **No hay marco generativo:** no podemos simular nuevos documentos desde el modelo.

Cuando usar LDA y cuando BERTopic

Puente: de
variables
latentes a
texto

Introducción:
datos no es-
tructurados

Análisis
textual:
Entender

Análisis
textual:
Estandarizar

Análisis
textual:
Describir

Análisis
textual:
Modelar con
LDA

LDA
Supervisado
(sLDA)

Más allá de
BOW:
modelos de
lenguaje

Entrenamiento
con
gradiente
descendente

Situación	Recomendación
Corpus con vocabulario especializado	LDA
Se requiere cuantificar incertidumbre	LDA
Documentos cortos (tweets, titulares)	BERTopic
Corpus en varios idiomas	BERTopic
Exploración rápida con sinónimos	BERTopic
Interpretabilidad probabilística	LDA / sLDA
Pocos datos (< 1000 docs)	LDA
Corpus masivo con jerga variable	BERTopic

Puente: de
variables
latentes a
texto

Introduccion:
datos no es-
tructurados

Analisis
textual:
Entender

Analisis
textual:
Estandarizar

Analisis
textual:
Describir

Analisis
textual:
Modelar con
LDA

LDA
Supervisado
(sLDA)

Mas alla de
BOW:
modelos de
lenguaje

Entrenamiento
con
gradiente
descendente

Comparaciones y cierre

Comparación: LDA vs. Word2Vec vs. Transformers

Aspecto	LDA	W2V / GloVe	Transformers
Representación	Proporciones de temas	Vector fijo por palabra	Vector contextual
Orden importa	No (BOW)	No (ventana)	Si (toda la frase)
Polisemia	No	No	Si
Entrenamiento	EM/Gibbs	SGD	SGD
Interpretable	Si (temas legibles)	Parcialmente	No directamente
Datos requeridos	Pocos	Muchos	Masivos
Variable latente	Temas discretos	Vector continuo	Representación densa
Uso principal	Descubrir temas	Similitud semántica	NLP avanzado

Ventajas y desventajas de LDA

Ventajas:

- Los temas son **interpretables**: se puede leer la lista de palabras de un tema y darle un nombre.
- Funciona con **pocos datos** en comparación con modelos neuronales.
- Marco probabilístico claro: se puede cuantificar la incertidumbre sobre la asignación de temas.
- Permite comparar documentos por su mezcla de temas.

Desventajas:

- Modelo BOW: ignora el orden y la gramática.
- El número de temas K es un hiperparámetro que el analista debe elegir (como k en K-means).
- Los temas pueden no ser coherentes si el corpus es pequeño o heterogéneo.
- No captura polisemia (una palabra, varios significados).

Ventajas y desventajas de los modelos neuronales

Ventajas:

- Capturan dependencias de largo alcance en el texto.
- Representaciones contextuales (Transformers): el mismo token tiene representaciones distintas según el contexto.
- Estado del arte en clasificación, traducción, generación y comprensión de texto.

Desventajas:

- Requieren datos masivos (millones de documentos) y recursos computacionales enormes para entrenamiento.
- **Caja negra:** los vectores aprendidos no tienen una interpretación directa como los temas de LDA.
- Fine-tuning necesario para tareas específicas.
- Riesgo de perpetuar sesgos del corpus de entrenamiento.

Cuando usar cada metodo

Situacion	Metodo recomendado
Descubrir temas en un corpus nuevo	LDA
Temas + variable respuesta disponible	sLDA
Similitud semantica entre documentos	W2V / GloVe
OOV frecuente (errores, morfologia)	FastText
Clasificacion de texto, datos suficientes	Fine-tune BERT
Muy pocos datos etiquetados	LDA o embeddings pre-entrenados
Interpretabilidad requerida	LDA o sLDA
Generacion de texto	GPT / modelos causales
Análisis exploratorio inicial	BOW + LDA

Puente: de
variables
latentes a
texto

Introduccion:
datos no es-
tructurados

Analisis
textual:
Entender

Analisis
textual:
Estandarizar

Analisis
textual:
Describir

Analisis
textual:
Modelar con
LDA

LDA
Supervisado
(sLDA)

Mas alla de
BOW:
modelos de
lenguaje

Entrenamiento
con
gradiente
descendente

Sintesis y cierre

Puente: de
variables
latentes a
texto

Introducción:
datos no es-
tructurados

Análisis
textual:
Entender

Análisis
textual:
Estandarizar

Análisis
textual:
Describir

Análisis
textual:
Modelar con
LDA

LDA
Supervisado
(sLDA)

Más allá de
BOW:
modelos de
lenguaje

Entrenamiento
con
gradiente
descendente

S1–S3: datos estructurados, relaciones entre variables.

S4: kernels y localidad, el puente hacia objetos generales.

S5: respuestas no continuas, GLM.

S6: variables latentes en datos estructurados (EM, PCA, ACM).

S7: variables latentes en datos **no estructurados** (texto).

El hilo común: siempre buscamos estructura oculta en los datos, ya sean números, categorías o palabras.

LDA es EM con temas. sLDA añade una respuesta. Los embeddings y transformers aprenden las variables latentes con redes neuronales en lugar de modelos probabilísticos.

Puente: de
variables
latentes a
texto

Introduccion:
datos no es-
tructurados

Analisis
textual:
Entender

Analisis
textual:
Estandarizar

Analisis
textual:
Describir

Analisis
textual:
Modelar con
LDA

LDA
Supervisado
(sLDA)

Mas alla de
BOW:
modelos de
lenguaje

Entrenamiento
con
gradiente
descendente

Tarea	Herramienta en R
Tokenizacion y limpieza	<code>tidytext::unnest_tokens()</code>
Analisis de sentimientos	<code>textdata::get_sentiments()</code>
n-gramas y grafos de bigramas	<code>igraph</code> , <code>ggraph</code>
Document-Term Matrix	<code>tidytext::cast_dtm()</code>
Ajuste LDA	<code>topicmodels::LDA()</code>
Extraer temas y proporciones	<code>tidytext::tidy()</code>
Embeddings estaticos	<code>word2vec</code> , <code>text (CRAN)</code>
Embeddings contextuales (BERT)	<code>reticulate</code> + <code>HuggingFace</code>
Modelos de lenguaje grandes	<code>reticulate</code> + <code>transformers</code>

Puente: de
variables
latentes a
texto

Introduccion:
datos no es-
tructurados

Analisis
textual:
Entender

Analisis
textual:
Estandarizar

Analisis
textual:
Describir

Analisis
textual:
Modelar con
LDA

LDA
Supervisado
(sLDA)

Mas alla de
BOW:
modelos de
lenguaje

Entrenamiento
con
gradiente
descendente

Referencias

Bibliografía

- (1) Blei, D.M., Ng, A.Y., Jordan, M.I. (2003). *Latent Dirichlet Allocation*. Journal of Machine Learning Research, 3, 993–1022.
- (2) Mcauliffe, J., Blei, D. (2007). *Supervised Topic Models*. Advances in Neural Information Processing Systems, 20.
- (3) Mikolov, T. et al. (2013). *Distributed Representations of Words and Phrases*. NeurIPS.
- (4) Pennington, J., Socher, R., Manning, C. (2014). *GloVe: Global Vectors for Word Representation*. EMNLP.
- (5) Peters, M. et al. (2018). *Deep Contextualized Word Representations (ELMo)*. NAACL.
- (6) Vaswani, A. et al. (2017). *Attention Is All You Need*. NeurIPS.
- (7) Silge, J., Robinson, D. *Text Mining with R: A Tidy Approach*. O'Reilly. Disponible en: <https://www.tidytextmining.com/>
- (8) Hastie, T., Tibshirani, R., Friedman, J. *The Elements of Statistical Learning*. Springer.

Paquetes en R: tidytext, topicmodels, gutenbergr, textdata, wordcloud, igraph, ggraph, ggrepel.